Criterion A: Planning

Scenario: The client, Bill, a friend of mine, is a recent high school graduate from Athens who will study medicine in a university in Thessaloniki[1]. He is provided with 1500 euros from his parents to assist him in handling the first two months of his stay in Thessaloniki and is planning to find a job. However, he fears that he will run out of money on unnecessary expenses, leaving him unable to pay his liabilities.

It is also mentioned that he does not trust computer programs and the Internet due to previous accounts of his being stolen[2], possibly due to the same password being utilized for multiple accounts and he generally has a fear of data loss

Solution and Rationale: A financial management application that will focus on the ability of Bill to create his own budget, which will consist of categories, subcategories, and each respective date and amount to be paid for each category/subcategory that will be stored in a database to categorize and classify his expenses. Also, considering the money given by his family, there needs to be a section devoted to deposits and withdrawals that would constitute an account balance which would improve the accuracy of the budgetary predictions immensely. Due to the lack of computer skills of my client[3], a user-friendly GUI should be implemented so Bill can select specific options using his cursor and buttons that call specific functions, thus limiting annoyance and enhancing usability. Bill also mentioned issues with accounts and data being stolen in the past, and thus a section should be devoted to account creation and log-in, as Bill does not want to share his password with me[4], which will be encompassed by password strength policies to protect his data, such as upper and lower case characters and password obfuscation[5]. Considering Bill frequently uses his gmail, budgetary data should also be sent to his

[1] See Appendix A
[2] See Appendix A
[3] See Appendix A
[4] See Appendix A
[5] See Appendix A

email based on command, to further mitigate his fear of losing and to better organize his data[6]. Financial projections regarding when his money will run out based on average spending and his account balance will also be provided, to help Bill manage his expenses and spending habits. Lastly, Bill can sort budgetary data (e.g category or amounts) based on his own preference and importance[7] and the database is to be saved on cloud storage as Bill is scared of losing his files.

I will use Python for front and back-end development and a database, it being SQLite, for the creation of the program. Python will be utilized as it offers a variety of libraries such as Matplotlib, which can be used to graph financial projections, and due to its higher level of abstraction, it has a large online community. As my user does not necessarily need a fancy GUI, and the application will not require heavy resources, Tkinter will be utilized as it offers portability and ease of use. SQLite will be used as it operates efficiently without a server, is lightweight, and allows for persistent storage of data, ideal considering Bill does not want to share personal data with me (password) and generally will not require large amounts of data to be stored. The database is needed to store budgetary data of Bill persistently in case the application shuts down and to store Bill's account information and budgetary data. Furthermore, for security, the database is to be saved on a Google Drive folder, using a Python API, which Bill will have access to and uses a lot[8], as Google Drive is popular and can easily be integrated with Google technologies, including Gmail.

Success Criteria:

1. Bill can deposit and withdraw money from his virtual account balance without any disruptions with the calculations or input.

2. Bill can access the history of his account and each data. Basically Bill can log in his account and access his previously created budget and account balance despite shutting down the application.

[6] See Appendix A
[7] See Appendix A
[8] See Appendix A

3. Bill can create new or delete existing budgetary data regarding categories, subcategories, dates of each category/subcategory to be paid, and the amount to be paid

4. Bill's password within the database is to be obfuscated

5. Bill has the choice to send his created budget to his gmail account

6. The sqlite database is to be constantly saved to a Google Drive folder using a Python API

7. Bill is able to sort budgetary data (e.g categories, subcategories, amounts to be paid, dates to be paid) either in ascending or descending order based on his preference and importance of each

8. Password strength policies (e.g. upper and lower case letters, special characters) for password creation are to be taken into consideration.

9. Projections of when Bill's money will run out based on his average spending, payment dates for each category/subcategory, and the corresponding amounts should be provided.

10. Graphs showcasing his spending trends against dates to be paid, amounts, and his average spending should also be provided and plotted based on Bill's command

11. Creation of an SQLite database where Bill's budgetary data and account information are to be updated and stored based on changes made within the application (e.g Bill adds a new category)

12. Any further extreme data, abnormal data, and exceptions are to be handled

Word Count: 574